JAVA PLACEMENT MASTER SERIES · HANDBOOK 2

Core Java Essentials

Exceptions, Strings, Generics, IO/NIO, Date-Time, Reflection & Annotations — the Java every interviewer probes

★ Exception Handling Mastery

★ Generics & PECS Explained

★ Reflection & Annotations

★ Top 75 Interview Q&A

For MCA / B.Tech / B.E. Placement Preparation

Java Developer · Backend Roles · Spring Boot Prep

Table of Contents

Tip: If you're short on time, revise **Part 17 (Final Revision)** and **Part 13 (Frequently Confused Concepts)** first — they compress the entire handbook into rapid-fire tables and a Top 75 Q&A list.

How to use this handbook

This is **Handbook 2** of the Java Placement Master Series, covering Core Java Essentials — the topics that come right after Fundamentals & OOP and show up constantly in interviews: exceptions, strings, generics, IO/NIO, date-time, regex, reflection, and annotations. Read it top to bottom once, then use **Part 17 (Final Revision)** as your fast recap before interviews.

Importance tags used throughout:

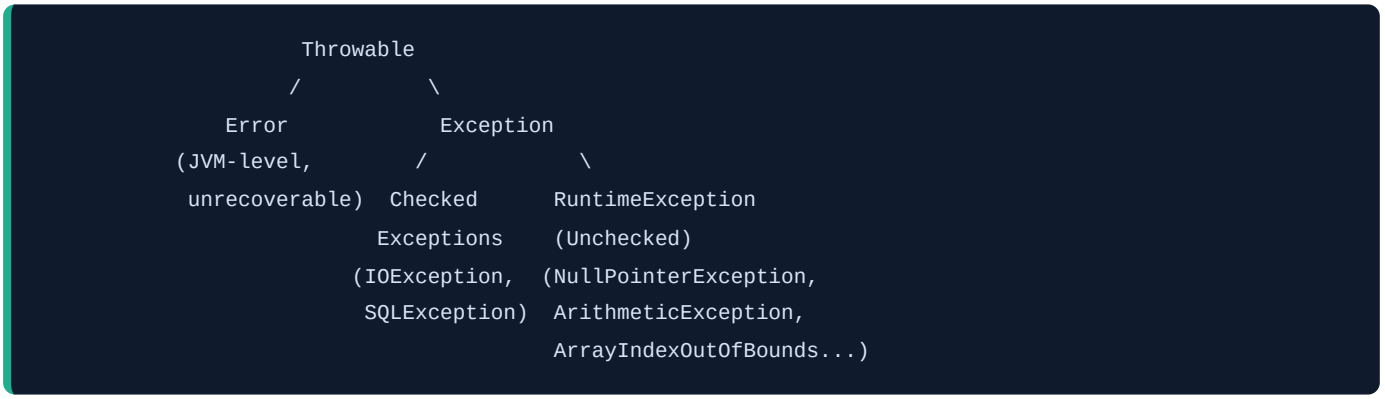
- ⭐⭐⭐⭐⭐ Must Know for Placements
- ⭐⭐⭐⭐ Frequently Asked
- ⭐⭐ Medium Importance

Part 1 — Exception Handling ⭐⭐⭐⭐⭐

Why Exceptions Exist

Without exceptions, error conditions (file not found, divide by zero, invalid input) would have to be checked manually with error codes after every operation — messy and easy to forget. Exceptions let Java **separate normal logic from error-handling logic**, and force problems to be handled explicitly rather than failing silently.

Exception Hierarchy



Type	Meaning	Examples	Must Handle?
Error	Serious JVM-level problems, app usually can't recover	`OutOfMemoryError`, `StackOverflowError`	No — not meant to be caught
Checked Exception	Anticipated conditions the compiler forces you to handle	`IOException`, `SQLException`	Yes — compile error if ignored
Unchecked Exception (RuntimeException)	Programming bugs, not enforced by the compiler	`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`	No — but should still be prevented

> ***Compile-time vs Runtime errors:*** *Compile-time errors are syntax/type mistakes caught by* `javac` *before the program runs. Runtime errors (exceptions) occur while the program is executing.*

try / catch / finally / throw / throws

```
try {
    int result = 10 / 0;              // may throw ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
    System.out.println("Always runs - cleanup code goes here");
}
```

Keyword	Purpose
`try`	Wraps code that might throw an exception
`catch`	Handles a specific exception type
`finally`	Always executes (cleanup), whether an exception occurred or not — even if `return` was hit in `try` / `catch`
`throw`	Manually throws an exception instance: `throw new IllegalArgumentException("bad input")`
`throws`	Declares that a method **might** throw a checked exception, in the method signature

```
void readFile(String path) throws IOException {   // declares checked exception
    throw new IOException("File not found");        // actually throws it
}
```

Multi-catch and Nested try ⭐⭐⭐

```
try {
    // risky code
} catch (IOException | SQLException e) {    // multi-catch (Java 7+) - handle both the same way
    System.out.println("Error: " + e.getMessage());
}

try {                                       // outer try
    try {                                    // nested try
        int[] arr = new int[5];
        arr[10] = 1;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Inner catch handled it");
    }
} catch (Exception e) {
    System.out.println("Outer catch - backup handler");
}
```

Exception Propagation ⭐⭐⭐⭐

If a method doesn't catch an exception, it **propagates up** the call stack to the caller, and keeps propagating until some method catches it — or the program crashes if nobody does.

```
methodC() throws exception
     |  (not caught here)
     ▼
methodB() called methodC()
     |  (not caught here either)
     ▼
methodA() called methodB()
     |  (finally caught here!)
     ▼
   catch block handles it
```

Custom Exceptions ⭐⭐⭐⭐

```
class InsufficientFundsException extends Exception {   // checked custom exception
    public InsufficientFundsException(String message) {
        super(message);
    }
}

void withdraw(double amount, double balance) throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException("Not enough balance");
    }
}
```

Best practice: Extend `RuntimeException` for programming-error-style custom exceptions (unchecked), or `Exception` for conditions callers are expected to anticipate and recover from (checked).

Try-With-Resources ⭐⭐⭐⭐⭐

Automatically closes any resource implementing `AutoCloseable` (e.g., file streams, DB connections) when the `try` block ends — **no need for a manual `finally` block to close resources.**

```
// Old way - verbose, error-prone
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("file.txt"));
    // use br
} finally {
    if (br != null) br.close();      // easy to forget or get wrong
}

// Try-with-resources (Java 7+) - resource auto-closed
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
}   // br.close() called automatically here, even if an exception occurs
```

Suppressed Exceptions ⭐⭐⭐

If both the `try` block body **and** the automatic `close()` call throw exceptions, the `close()`'s exception is **suppressed** (attached to the main exception, retrievable via `getSuppressed()`) rather than replacing the original — so the root cause isn't lost.

Best Practices & Common Mistakes

Mistake	Why It's Bad	Fix
Catching `Exception` (or worse, `Throwable`) everywhere	Hides real bugs, catches things you didn't intend to (like `Error`s)	Catch the most **specific** exception type possible
Swallowing exceptions (`catch (Exception e) {}`)	Silently hides failures — extremely hard to debug in production	At minimum, log the exception; never leave a catch block empty
Using exceptions for normal control flow	Exceptions are relatively expensive (stack trace capture) and hurt readability	Use `if` checks for expected conditions; reserve exceptions for truly exceptional cases
Not closing resources	Causes resource/memory leaks (file handles, DB connections)	Always use try-with-resources

Performance consideration ⭐⭐⭐: Creating an exception captures the full **stack trace**, which has a real (though usually small) performance cost. Avoid throwing exceptions in hot loops/performance-critical paths for routine, expected conditions.

Interview Questions — Part 1

Q1. Difference between checked and unchecked exceptions? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Checked exceptions (subclasses of `Exception`, excluding `RuntimeException`) are checked by the compiler at compile time — the method must either handle or declare them with `throws`. Unchecked exceptions (subclasses of `RuntimeException`) represent programming bugs and aren't enforced by the compiler.
- **Why asked:** Foundational — tests basic exception hierarchy understanding.
- **Common mistake:** Saying "checked exceptions happen at compile time" — they still happen at runtime; only the *requirement to handle them* is enforced at compile time.
- **Difficulty:** Basic

Q2. Difference between `throw` and `throws`? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `throw` actually throws an exception instance at a specific point in code. `throws` is used in a method signature to declare that the method might propagate a checked exception to its caller.
- **Difficulty:** Basic

Q3. Does `finally` always execute? Are there exceptions? ⭐⭐⭐⭐

- **Ideal Answer:** `finally` runs even if `try` / `catch` has a `return` statement. The rare exceptions are if `System.exit()` is called, or the JVM crashes/is killed before `finally` runs.
- **Why asked:** Tests deeper understanding of control flow, not just the textbook rule.
- **Difficulty:** Intermediate

Q4. How does try-with-resources work internally? ⭐⭐⭐⭐

- **Ideal Answer:** Any resource implementing `AutoCloseable` declared in the `try(...)` parentheses has its `close()` method automatically called when the block exits — in reverse order of declaration — whether the block completes normally or via exception, eliminating the need for manual `finally`-based cleanup.
- **Difficulty:** Intermediate

Q5. Scenario: A `try` block throws an exception, and the `finally` block also throws one. Which one propagates? ⭐⭐⭐⭐

- **Ideal Answer:** The exception from the `finally` block wins and propagates to the caller — the original exception from `try` is **lost** (unless manually captured). This is a classic reason to keep `finally` blocks simple and avoid throwing from them.
- **Why asked:** Tricky edge-case question testing real depth.
- **Difficulty:** Advanced

Part 2 — Strings ⭐⭐⭐⭐⭐

Strings Are Immutable — Why?

Once a `String` object is created, its content **can never change**. Any operation that looks like it "modifies" a string actually creates a **new** `String` object.

```
String s = "Hello";
s.concat(" World");     // creates a NEW string, doesn't change 's'
System.out.println(s);  // still prints "Hello"

s = s.concat(" World");  // must REASSIGN to capture the new string
System.out.println(s);   // now prints "Hello World"
```

Why Strings are immutable ⭐⭐⭐⭐⭐:

1. **String Pool optimization** — immutability makes it safe for multiple references to share the same object (see below)
2. **Thread safety** — immutable objects can be shared across threads with no synchronization needed
3. **Security** — Strings are used for class names, file paths, network connections; if mutable, malicious code could alter a string after a security check but before it's used
4. **Hashcode caching** — since content never changes, `String`'s `hashCode()` can be computed once and cached, making `HashMap` lookups with String keys fast

The String Pool ⭐⭐⭐⭐⭐

The **String Pool** (aka String Constant Pool) is a special memory region (part of the heap since Java 7) where **string literals** are stored and **reused**.

```
String a = "Java";        // literal - checks pool first, reuses if found
String b = "Java";        // literal - "Java" already in pool, reuses SAME object
String c = new String("Java");   // FORCES a new object on heap, bypasses the pool

System.out.println(a == b);   // true  - both point to the SAME pooled object
System.out.println(a == c);   // false - 'c' is a separate heap object
System.out.println(a.equals(c)); // true - content is equal
```

String Interning ⭐⭐⭐

```
String c = new String("Java").intern();   // manually forces 'c' into the pool
System.out.println(a == c);                // now true - both reference the pooled object
```

`.intern()` checks if an equal string already exists in the pool; if so, returns the pooled reference, otherwise adds this string to the pool. This is a memory optimization technique for programs that create many duplicate strings.

Common String Methods

Method	Purpose
`length()`	Number of characters
`charAt(i)`	Character at index `i`
`substring(start, end)`	Extract a portion
`indexOf(str)`	First index of a substring (-1 if not found)
`equals()` / `equalsIgnoreCase()`	Content comparison
`compareTo()`	Lexicographic comparison
`trim()` / `strip()`	Remove leading/trailing whitespace (`strip()` is Unicode-aware, Java 11+)
`split(regex)`	Split into a `String[]`
`replace()`	Replace characters/substrings
`toUpperCase()` / `toLowerCase()`	Case conversion
`format()`	Formatted string creation, similar to `printf`

StringBuilder and StringBuffer ⭐⭐⭐⭐⭐

Since `String` is immutable, repeatedly modifying strings (e.g., in a loop) creates **many throwaway objects** — wasteful. `StringBuilder` / `StringBuffer` are **mutable** character sequences designed for efficient, repeated modification.

```
// BAD - creates a new String object on every iteration (O(n²) overall)
String result = "";
for (int i = 0; i < 1000; i++) {
    result += i;      // each += creates a new String object!
}

// GOOD - modifies the same internal buffer (O(n) overall)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i);        // modifies internal char array, no new object per iteration
}
String result2 = sb.toString();
```

	`String`	`StringBuilder`	`StringBuffer`
Mutability	Immutable	Mutable	Mutable
Thread Safety	Immutable → inherently safe	**Not** synchronized	**Synchronized** (thread-safe)
Performance	Slow for repeated modification	Fast (single-threaded)	Slower than StringBuilder (sync overhead)
When to use	Fixed/rarely-changing text	Building strings in a single thread (most common case)	Building strings shared across multiple threads

Output Prediction Example ⭐⭐⭐⭐

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");
String s4 = s3.intern();

System.out.println(s1 == s2);   // true  - both from pool
System.out.println(s1 == s3);   // false - s3 is a separate heap object
System.out.println(s1 == s4);   // true  - intern() returns the pooled reference
```

Interview Questions — Part 2

Q1. Why are Strings immutable in Java? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** For String Pool safety (shared references can't be corrupted), thread safety (no synchronization needed), security (strings used in sensitive contexts like class loading/file paths can't be altered after validation), and to allow caching the hashcode for fast HashMap lookups.
- **Difficulty:** Intermediate

Q2. Difference between `String s = "abc"` and `String s = new String("abc")`? ⭐⭐⭐⭐⭐

- **Ideal Answer:** The literal form checks the String Pool first and reuses an existing object if present. `new String("abc")` always creates a new object on the heap, bypassing the pool, even if `"abc"` already exists there.
- **Why asked:** Extremely common — tests understanding of String Pool mechanics.
- **Difficulty:** Intermediate

Q3. When should you use StringBuilder instead of String concatenation? ⭐⭐⭐⭐⭐

- **Ideal Answer:** When building/modifying strings repeatedly, especially inside loops — String concatenation creates a new object on every operation (costly), while StringBuilder modifies an internal mutable buffer.
- **Difficulty:** Basic

Q4. StringBuilder vs StringBuffer? ⭐⭐⭐⭐

- **Ideal Answer:** Both are mutable, but StringBuffer's methods are synchronized (thread-safe, slightly slower), while StringBuilder is not synchronized (faster, but not safe for concurrent modification from multiple threads).
- **Difficulty:** Basic

Part 3 — Arrays ⭐⭐⭐⭐

Single and Multi-Dimensional Arrays

```
int[] nums = {1, 2, 3, 4, 5};                // single-dimensional
int[][] matrix = {{1, 2}, {3, 4}, {5, 6}};     // 2D array (3 rows, 2 cols)
int[][] jagged = new int[3][];                  // jagged array - rows can have different lengths
jagged[0] = new int[]{1};
jagged[1] = new int[]{1, 2, 3};
jagged[2] = new int[]{1, 2};
```

```
Regular 2D array (rectangular):     Jagged array (uneven rows):
[1, 2]                              [1]
[3, 4]                              [1, 2, 3]
[5, 6]                              [1, 2]
```

Arrays are **fixed-size** once created and are **objects** in Java (stored on the heap), with a public `length` field (not a method — no parentheses).

The `Arrays` Utility Class ⭐⭐⭐⭐

Method	Purpose
`Arrays.sort(arr)`	Sorts in place (Dual-Pivot Quicksort for primitives, Timsort for objects)
`Arrays.binarySearch(arr, key)`	Fast search — **array must be sorted first**
`Arrays.copyOf(arr, newLength)`	Returns a new, resized copy
`Arrays.copyOfRange(arr, from, to)`	Copies a sub-range
`Arrays.equals(a, b)`	Shallow comparison — element-by-element for 1D arrays
`Arrays.deepEquals(a, b)`	Recursively compares nested arrays (needed for 2D+ arrays)
`Arrays.fill(arr, value)`	Fills all elements with a given value
`Arrays.toString(arr)` / `Arrays.deepToString(arr)`	Readable string representation
`Arrays.asList(arr)`	Fixed-size `List` view backed by the array

```
int[][] a = {{1,2},{3,4}};
int[][] b = {{1,2},{3,4}};
System.out.println(Arrays.equals(a, b));      // false! compares references of inner arrays
System.out.println(Arrays.deepEquals(a, b));   // true - recursively compares contents
```

Copying Arrays — Shallow vs Deep ⭐⭐⭐⭐

```
int[] original = {1, 2, 3};
int[] copy = original.clone();          // shallow copy for primitives - values duplicated

// For arrays of OBJECTS, clone()/copyOf() is shallow -
// the new array holds the SAME object references, not deep copies of the objects
String[] names = {"A", "B"};
String[] namesCopy = names.clone();     // new array, but same String references inside
```

`System.arraycopy()` — the fastest, lowest-level way to copy array data (used internally by `Arrays.copyOf`):

```
int[] src = {1, 2, 3, 4, 5};
int[] dest = new int[5];
System.arraycopy(src, 0, dest, 0, 5);   // (source, srcPos, dest, destPos, length)
```

Common Interview Tricks ⭐⭐⭐⭐

```
int[] arr = new int[5];
System.out.println(arr[0]);          // 0 - numeric arrays default to 0

String[] strArr = new String[5];
System.out.println(strArr[0]);       // null - object arrays default to null

int[] a = {1, 2, 3};
int[] b = a;                          // 'b' points to the SAME array object!
b[0] = 99;
System.out.println(a[0]);             // 99 - 'a' was modified too (same reference)
```

Performance: Array vs ArrayList (brief preview)

Arrays have **fixed size** and store **primitives directly** (fast, low memory overhead). `ArrayList` is resizable but stores **objects** (requires autoboxing for primitives, extra memory per element). Use arrays when size is fixed and performance is critical; use `ArrayList` for flexibility.

Interview Questions — Part 3

Q1. What is the default value of array elements? ⭐⭐⭐⭐

- **Ideal Answer:** Numeric primitive arrays default to `0` (or `0.0`), `boolean` arrays default to `false`, and object-reference arrays (including `String[]`) default to `null`.
- **Difficulty:** Basic

Q2. Difference between `Arrays.equals()` and `Arrays.deepEquals()`? ⭐⭐⭐⭐

- **Ideal Answer:** `Arrays.equals()` performs a shallow, element-by-element comparison — for a 2D array, it compares inner array *references*, not their contents. `Arrays.deepEquals()` recursively compares nested array contents, needed for correctly comparing multi-dimensional arrays.
- **Difficulty:** Intermediate

Q3. Is array cloning a deep copy? ⭐⭐⭐⭐

- **Ideal Answer:** No — `clone()` performs a shallow copy. For primitive arrays this is effectively a full copy of values, but for object arrays, the cloned array contains the same object references as the original, not independent copies of the objects themselves.
- **Difficulty:** Intermediate

Part 4 — Wrapper Classes ⭐⭐⭐⭐

Why Wrapper Classes Exist

Java's collections (`ArrayList` , `HashMap` , etc.) and generics work only with **objects**, not primitives. Wrapper classes (`Integer` , `Double` , `Boolean` , etc.) give every primitive an object equivalent so they can be used wherever objects are required — like in a `List<Integer>` .

Primitive	Wrapper Class
`int`	`Integer`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`
`long`	`Long`
`byte`	`Byte`
`short`	`Short`
`float`	`Float`

Autoboxing and Unboxing

```
Integer boxed = 10;         // Autoboxing: int → Integer (compiler inserts Integer.valueOf(10))
int unboxed = boxed;         // Unboxing: Integer → int (compiler inserts boxed.intValue())

List<Integer> list = new ArrayList<>();
list.add(5);                  // 5 is autoboxed to Integer.valueOf(5)
int x = list.get(0);          // Integer is unboxed to int
```

Integer Caching ⭐⭐⭐⭐⭐

Java caches `Integer` objects for values from **-128 to 127** (`Integer.valueOf()` reuses cached instances in this range instead of creating new objects, as a performance optimization since small integers are extremely common).

```
Integer a = 100, b = 100;
System.out.println(a == b);     // true - both from the cache

Integer c = 200, d = 200;
System.out.println(c == d);     // false - outside cache range, different objects

System.out.println(c.equals(d));  // true - always use .equals() for wrapper value comparison
```

This same caching behavior applies to `Byte`, `Short`, `Long`, `Character` (for small values) and `Boolean`.

Parsing and Utility Methods

```
int i = Integer.parseInt("42");        // String → primitive int
Integer obj = Integer.valueOf("42");    // String → Integer object (uses cache if in range)
String s = Integer.toString(42);        // int → String
int max = Integer.MAX_VALUE;             // 2147483647
int min = Integer.MIN_VALUE;             // -2147483648
boolean isPrime = ...;                    // (utility logic would go here)
```

Performance Implications & NullPointerException Pitfalls ⭐⭐⭐⭐⭐

- Autoboxing/unboxing has overhead (object creation, method calls) — avoid in performance-critical, tight loops where primitives would suffice.
- **Classic pitfall:** unboxing a `null` wrapper throws `NullPointerException`.

```
Integer count = null;
int x = count;              // NullPointerException! unboxing null fails

Map<String, Integer> scores = new HashMap<>();
int score = scores.get("Alice");   // NPE if "Alice" isn't in the map (get returns null)
```

Interview Questions — Part 4

Q1. Explain Integer caching with an example. ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Java caches `Integer` objects for values -128 to 127 via `Integer.valueOf()`. So `Integer a=100,b=100; a==b` is `true` (same cached object), but for 200 it's `false` (new objects created outside the cache range). Always use `.equals()` to compare wrapper values reliably.
- **Difficulty:** Intermediate

Q2. Why does unboxing a null Integer throw NullPointerException? ⭐⭐⭐⭐

- **Ideal Answer:** Unboxing calls a method like `.intValue()` on the wrapper object; if the reference is `null`, calling any method on it throws NPE. This commonly happens with `Map.get()` returning `null` for a missing key, then being auto-unboxed into a primitive.
- **Difficulty:** Intermediate

Part 5 — Enums ⭐⭐⭐⭐

Why Enums Exist

Before enums, developers used `int` or `String` constants to represent a fixed set of values (e.g., days of the week), which had no type safety — any `int` could be passed where a "day" was expected. **Enums** provide a **type-safe, fixed set of named constants.**

```
// Old, unsafe way
final int MONDAY = 0, TUESDAY = 1;
void setDay(int day) { ... }     // nothing stops setDay(999)!

// Enum way - type safe
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }
void setDay(Day day) { ... }      // only valid Day values are possible
```

Enums with Constructors, Fields, and Methods ⭐⭐⭐⭐

Enums in Java are actually **full classes** — they can have constructors, fields, and methods (they implicitly extend `java.lang.Enum`).

```
enum Planet {
    MERCURY(3.3e23, 2.4e6),
    EARTH(5.9e24, 6.4e6);

    private final double mass;      // fields
    private final double radius;

    Planet(double mass, double radius) {   // constructor - called once per constant
        this.mass = mass;
        this.radius = radius;
    }

    double surfaceGravity() {          // method
        return 6.67e-11 * mass / (radius * radius);
    }
}

System.out.println(Planet.EARTH.surfaceGravity());
```

switch with Enum

```
Day today = Day.MONDAY;
switch (today) {
    case MONDAY -> System.out.println("Start of the week");
    case FRIDAY -> System.out.println("Almost weekend");
    default -> System.out.println("Midweek");
}
```

EnumSet and EnumMap ⭐⭐⭐

- **`EnumSet`** — a highly efficient `Set` implementation designed specifically for enum types (implemented internally using bit vectors — extremely fast).
- **`EnumMap`** — a `Map` implementation with enum keys, internally backed by an array indexed by the enum's ordinal — faster than `HashMap` for enum keys.

```
EnumSet<Day> weekend = EnumSet.of(Day.SATURDAY, Day.SUNDAY);
EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
schedule.put(Day.MONDAY, "Team meeting");
```

Best Practices & Real-World Examples

- Use enums for any fixed, known set of values: `OrderStatus { PENDING, SHIPPED, DELIVERED, CANCELLED }`, `Role { ADMIN, USER, GUEST }`, `HttpStatus`, direction constants, etc.
- Prefer enums over `int` / `String` constants whenever the set of values is fixed and known at compile time — you get type safety, `switch` support, and the ability to attach behavior.

Interview Questions — Part 5

Q1. Why use enums instead of int/String constants? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Enums provide compile-time type safety (invalid values can't be passed), can hold their own fields/methods/constructors, integrate with `switch`, and produce more readable, self-documenting code than raw magic numbers or strings.
- **Difficulty:** Basic

Q2. Can an enum have a constructor? Can it be public? ⭐⭐⭐⭐

- **Ideal Answer:** Yes, enums can have constructors, but they must be `private` (or package-private) — never `public` — because enum constants are the only instances ever created, and they're all constructed internally when the enum type is loaded.
- **Difficulty:** Intermediate

Q3. What is EnumSet and why is it efficient? ⭐⭐⭐

- **Ideal Answer:** `EnumSet` is a specialized `Set` for enum types, internally implemented as a bit vector where each bit represents whether a constant is present — making operations extremely fast and memory-compact compared to a general-purpose `HashSet`.
- **Difficulty:** Advanced

Part 6 — Generics

Why Generics Were Introduced

Before generics (pre-Java 5), collections stored `Object` — requiring manual casting and offering **no compile-time type safety**.

```
// Pre-generics - unsafe
List list = new ArrayList();
list.add("Hello");
list.add(42);                       // no error! mixed types allowed
String s = (String) list.get(1);     // ClassCastException at RUNTIME - too late!

// With Generics - type safe
List<String> list2 = new ArrayList<>();
list2.add("Hello");
// list2.add(42);                    // COMPILE ERROR - caught immediately
String s2 = list2.get(0);             // no cast needed
```

Type erasure aside, the core benefit is: **catch type mistakes at compile time instead of runtime.**

Generic Classes and Methods

```
class Box<T> {                          // generic class - T is a type parameter
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}

Box<String> stringBox = new Box<>();
stringBox.set("Hello");

// Generic method - type parameter declared before the return type
<T> void printArray(T[] array) {
    for (T item : array) System.out.println(item);
}
```

Generic Interfaces

```
interface Repository<T, ID> {
    T findById(ID id);
    void save(T entity);
}
class UserRepository implements Repository<User, Long> {
    public User findById(Long id) { ... }
    public void save(User entity) { ... }
}
```

This exact pattern is how **Spring Data JPA's** `JpaRepository<T, ID>` works.

Bounded Types ⭐⭐⭐⭐

Restrict what types can be used as a type argument.

```
class NumberBox<T extends Number> {    // T must be Number or a subclass (Integer, Double...)
    T value;
    double doubled() { return value.doubleValue() * 2; }   // safe - Number guarantees doubleValue()
}
// NumberBox<String> box;   // COMPILE ERROR - String is not a Number
```

Wildcards and PECS ⭐⭐⭐⭐⭐

Wildcard	Meaning	Use Case
`<?>`	Unknown type	When you don't care about the specific type
`<? extends T>`	**Upper bound** — T or any subtype	**Producer** — reading/getting values out
`<? super T>`	**Lower bound** — T or any supertype	**Consumer** — writing/putting values in

PECS mnemonic: "Producer Extends, Consumer Super" ⭐⭐⭐⭐⭐

```
void printAll(List<? extends Number> list) {   // PRODUCER - just reading
    for (Number n : list) System.out.println(n);
    // list.add(5);   // COMPILE ERROR - can't safely add to an "extends" list
}

void addNumbers(List<? super Integer> list) {   // CONSUMER - just writing
    list.add(1);
    list.add(2);
    // Integer x = list.get(0);   // only safe to read as Object, not Integer
}
```

Why this restriction exists: With `List<? extends Number>`, the list could actually be a `List<Integer>` or `List<Double>` — the compiler can't guarantee what specific type is safe to *add*, so it disallows adding entirely (except `null`). With `List<? super Integer>`, the list is guaranteed to accept `Integer` (since it's `Integer` or some supertype), so adding is safe, but reading only guarantees `Object`.

Type Erasure ⭐⭐⭐⭐⭐

Generics are a **compile-time-only** feature — the compiler checks types, then **erases** them, replacing generic types with `Object` (or the bound type) in the compiled bytecode. This is why generic type information doesn't exist at runtime.

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();
System.out.println(strings.getClass() == ints.getClass());   // true! Both are just ArrayList at runtime
```

Consequences of type erasure:

- You **cannot** create a generic array directly: `T[] arr = new T[10];` → compile error (**Generic Arrays** restriction)
- You **cannot** use `instanceof` with a parameterized type: `if (obj instanceof List<String>)` → compile error
- Overloading methods that differ only by generic type parameter is not allowed (they erase to the same signature)

Heap Pollution ⭐⭐⭐

Occurs when a variable of a parameterized type refers to an object that is **not** of that parameterized type — typically via unchecked casts or raw type mixing, leading to a `ClassCastException` later, often far from the actual cause.

```
List<String> stringList = new ArrayList<>();
List rawList = stringList;              // raw type - unchecked warning
rawList.add(42);                          // heap pollution! an Integer snuck into a List<String>
String s = stringList.get(0);              // ClassCastException here, far from the real bug
```

Interview Questions — Part 6

Q1. Why were Generics introduced in Java? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** To provide compile-time type safety for collections and other classes, eliminating the need for manual casting and catching type mismatches at compile time instead of causing a `ClassCastException` at runtime.
- **Difficulty:** Basic

Q2. Explain PECS with an example. ⭐⭐⭐⭐⭐

- **Ideal Answer:** "Producer Extends, Consumer Super" — use `? extends T` when you only read values out of a structure (it "produces" values for you), and `? super T` when you only write values into it (it "consumes" values from you). This is because `extends` wildcards can't guarantee safe writes, while `super` wildcards can't guarantee a specific read type beyond `Object`.
- **Why asked:** A very common "prove you actually understand generics" question, often via the `Collections.copy(dest, src)` signature.
- **Difficulty:** Advanced

Q3. What is type erasure and what are its consequences? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Generic type information exists only at compile time; the compiler erases it and substitutes `Object` (or the bound) in the bytecode. Consequences: you can't create generic arrays directly, can't use `instanceof` with parameterized types, and two overloads differing only by generic parameter aren't allowed.
- **Why asked:** Tests deep JVM/compiler understanding, common in senior interviews.
- **Difficulty:** Advanced

Q4. What is heap pollution? ⭐⭐⭐

- **Ideal Answer:** When an object of the wrong parameterized type ends up inside a generic structure — usually via raw types or unchecked casts — leading to a `ClassCastException` at a point far removed from where the actual bad insertion happened.
- **Difficulty:** Advanced

Part 7 — File Handling (java.io) ⭐⭐⭐⭐

The `File` Class

Represents a file/directory **path** — not the actual content. Used for metadata operations (exists, delete, mkdir, list files), not reading/writing data itself.

```
File f = new File("data.txt");
System.out.println(f.exists());
System.out.println(f.length());
f.delete();
```

Byte Streams vs Character Streams ⭐⭐⭐⭐

	Byte Streams	Character Streams
Base classes	`InputStream` / `OutputStream`	`Reader` / `Writer`
Handles	Raw binary data (images, video, any file)	Text data, with proper character encoding
Example classes	`FileInputStream`, `FileOutputStream`	`FileReader`, `FileWriter`, `BufferedReader`

Reading and Writing Text Files

```
// Reading - BufferedReader wraps FileReader for efficient line-by-line reading
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}

// Writing - BufferedWriter for efficient writes; PrintWriter adds convenient print methods
try (PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter("out.txt")))) {
    pw.println("Hello, file!");
    pw.printf("Score: %d%n", 95);
}

// Scanner - convenient for reading formatted/mixed input (also used for console input)
try (Scanner sc = new Scanner(new File("data.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
}
```

Why `Buffered*` wrappers matter ⭐⭐⭐⭐: Reading/writing one character or line at a time directly from disk is slow (each call may hit the OS). `BufferedReader` / `BufferedWriter` batch reads/writes into memory, drastically reducing the number of actual I/O operations.

Serialization and Deserialization ⭐⭐⭐⭐⭐

Serialization converts an object into a byte stream (to save to disk or send over a network). **Deserialization** reconstructs the object from that byte stream.

```
class User implements Serializable {   // marker interface - required to be serializable
    private static final long serialVersionUID = 1L;   // version control for compatibility
    String name;
    transient String password;          // 'transient' fields are SKIPPED during serialization
}

// Serialize
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
    oos.writeObject(new User());
}
// Deserialize
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser"))) {
    User u = (User) ois.readObject();
}
```

- `Serializable` — a marker interface (no methods) that flags a class as eligible for serialization.
- `transient` — excludes a field from serialization (e.g., passwords, cached/derived data, non-serializable fields).
- `serialVersionUID` — a version identifier; if it doesn't match between the serialized data and the current class definition, deserialization fails with `InvalidClassException` — protects against silently loading incompatible data.
- **Externalization** (`Externalizable` interface) — gives the developer **full manual control** over the serialization format (via `writeExternal()` / `readExternal()`), used for performance optimization or custom formats, at the cost of more code.

Interview Questions — Part 7

Q1. What is the purpose of `transient` ? ⭐⭐⭐⭐⭐

- **Ideal Answer:** Marks a field to be excluded from serialization — commonly used for sensitive data (passwords), derived/cacheable fields, or fields referencing non-serializable objects.
- **Difficulty:** Intermediate

Q2. What happens if `serialVersionUID` doesn't match during deserialization? ⭐⭐⭐⭐

- **Ideal Answer:** An `InvalidClassException` is thrown — this protects against loading serialized data that's incompatible with the current version of the class structure.
- **Difficulty:** Intermediate

Q3. Why use BufferedReader instead of reading a file character by character? ⭐⭐⭐⭐

- **Ideal Answer:** Each unbuffered read may involve an actual system call to the OS, which is slow. BufferedReader reads larger chunks into an internal memory buffer, drastically reducing the number of real I/O operations.
- **Difficulty:** Intermediate

Part 8 — Java NIO ⭐⭐⭐

Why NIO Was Introduced

Traditional I/O (`java.io`) is **stream-based and blocking** — a thread reading from a stream is stuck waiting until data arrives. **NIO** (New I/O, `java.nio`) introduced **buffer-based, non-blocking** I/O, better suited for high-performance and scalable applications (e.g., handling thousands of network connections).

	Traditional IO	NIO
Model	Stream-oriented	Buffer + Channel-oriented
Blocking	Always blocking	Supports non-blocking mode
Best for	Simple file reads/writes	High-throughput, scalable I/O (servers)

Path, Paths, Files ⭐⭐⭐⭐

Modern NIO.2 (Java 7+) file API — cleaner and more powerful than the old `File` class.

```
Path path = Paths.get("data.txt");            // or Path.of("data.txt") in Java 11+
System.out.println(Files.exists(path));
System.out.println(Files.size(path));

List<String> lines = Files.readAllLines(path);   // reads entire file into a List<String>
Files.writeString(path, "Hello NIO!");            // Java 11+ - simple string write

try (Stream<String> lineStream = Files.lines(path)) {   // memory-efficient line streaming
    lineStream.filter(l -> l.contains("error")).forEach(System.out::println);
}
```

Channels and Buffers ⭐⭐⭐

- **Channel** — represents an open connection to an I/O source (file, socket) — data flows through it in both directions.
- **Buffer** — a fixed-size block of memory used to read from/write to a channel in batches.

```
try (FileChannel channel = FileChannel.open(path, StandardOpenOption.READ)) {
    ByteBuffer buffer = ByteBuffer.allocate(1024);
    channel.read(buffer);
}
```

Interview Importance ⭐⭐⭐

NIO is less commonly asked in-depth for entry-level roles, but knowing the **conceptual difference** (blocking streams vs non-blocking buffer/channel model) and being comfortable with the modern `Path` / `Files` API (which has largely replaced `File` in new code) is expected.

Interview Questions — Part 8

Q1. Why was NIO introduced when java.io already existed? ⭐⭐⭐⭐

- **Ideal Answer:** Traditional I/O is stream-based and always blocking, which doesn't scale well for applications handling many concurrent connections. NIO introduced buffer/channel-based I/O with support for non-blocking operations, suited for high-throughput, scalable systems.
- **Difficulty:** Intermediate

Q2. What is the difference between `Files.readAllLines()` and `Files.lines()`? ⭐⭐⭐

- **Ideal Answer:** `readAllLines()` loads the entire file into memory as a `List<String>`. `Files.lines()` returns a lazily-evaluated `Stream<String>`, which is far more memory-efficient for large files since lines are processed one at a time rather

than all loaded at once.

- **Difficulty:** Intermediate

Part 9 — Date and Time API

Problems with the Old `Date` Class

The legacy `java.util.Date` (and `Calendar`) had major design flaws:

- **Mutable** — not thread-safe, dangerous to share across threads
- Months are **zero-indexed** (`0` = January) — a classic source of bugs
- Poor API design — mixing date, time, and timezone concerns into one class
- No clear distinction between a **date**, a **time**, and a **timestamp**

Java 8 introduced `java.time` (**JSR-310**), which is **immutable, thread-safe, and far more intuitive.**

Core Classes

Class	Represents
`LocalDate`	A date without time or timezone (`2025-06-15`)
`LocalTime`	A time without date or timezone (`14:30:00`)
`LocalDateTime`	Date + time, no timezone
`Instant`	A precise moment on the UTC timeline (machine timestamp)
`ZonedDateTime`	Date + time + timezone
`Duration`	A time-based amount (hours, minutes, seconds) — for `Instant` / `LocalTime`
`Period`	A date-based amount (years, months, days) — for `LocalDate`
`ZoneId`	A timezone identifier (`"Asia/Kolkata"`)
`DateTimeFormatter`	Formats/parses date-time objects to/from strings

```
LocalDate today = LocalDate.now();
LocalDate birthday = LocalDate.of(2000, 6, 15);          // month is 1-indexed here (unlike old Date!)
LocalDateTime meeting = LocalDateTime.of(2025, 6, 15, 14, 30);

Period age = Period.between(birthday, today);
System.out.println(age.getYears() + " years old");

Duration duration = Duration.between(LocalTime.of(9,0), LocalTime.of(17,30));
System.out.println(duration.toHours() + " hours");

ZonedDateTime ist = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String formatted = today.format(fmt);
LocalDate parsed = LocalDate.parse("15-06-2025", fmt);
```

Comparison with Old Date API

	`java.util.Date`	`java.time` (Java 8+)
Mutability	Mutable	Immutable
Thread safety	Not thread-safe	Thread-safe
Month indexing	0-indexed (confusing)	1-indexed (intuitive)
Separation of concerns	One class for everything	Separate classes: `LocalDate`, `LocalTime`, `ZonedDateTime`, etc.

Interview Questions — Part 9

Q1. Why is `java.time` preferred over `java.util.Date`? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** `java.time` classes are immutable and thread-safe (unlike mutable `Date`), have intuitive 1-indexed months, and cleanly separate concerns (date-only, time-only, date+time, date+time+zone) instead of cramming everything into one class.
- **Difficulty:** Basic

Q2. Difference between `Duration` and `Period`? ⭐⭐⭐⭐

- **Ideal Answer:** `Duration` measures time-based amounts (hours, minutes, seconds, nanoseconds) — used with time-focused types like `Instant` / `LocalTime`. `Period` measures date-based amounts (years, months, days) — used with `LocalDate`.
- **Difficulty:** Intermediate

Q3. What is the difference between `LocalDateTime` and `ZonedDateTime`? ⭐⭐⭐

- **Ideal Answer:** `LocalDateTime` has no timezone information at all — it's just a date+time as read on a wall clock. `ZonedDateTime` includes an explicit `ZoneId`, correctly representing a specific instant relative to a timezone (accounting for offsets/DST).
- **Difficulty:** Intermediate

Part 10 — Regular Expressions

Pattern and Matcher

```
Pattern pattern = Pattern.compile("[a-z]+");   // compile the regex once (reusable, efficient)
Matcher matcher = pattern.matcher("hello123world");

while (matcher.find()) {
    System.out.println(matcher.group());   // prints "hello", then "world"
}
```

Common Regex Syntax Reference

Symbol	Meaning
`.`	Any single character
`*`	0 or more of the preceding element
`+`	1 or more of the preceding element
`?`	0 or 1 of the preceding element
`\d`	Any digit `[0-9]`
`\w`	Word character `[a-zA-Z0-9_]`
`\s`	Whitespace character
`^` / `$`	Start / end of string
`[abc]`	Any one of a, b, c
`{n,m}`	Between n and m repetitions
`|`	OR

Real-World Validation Examples ⭐⭐⭐⭐⭐

```
// Email validation
String emailRegex = "^[\\w.+-]+@[\\w-]+\\.[a-zA-Z]{2,}$";
boolean validEmail = "user@example.com".matches(emailRegex);   // true

// Indian phone number (10 digits)
String phoneRegex = "^[6-9]\\d{9}$";
boolean validPhone = "9876543210".matches(phoneRegex);          // true

// Password: min 8 chars, at least 1 digit, 1 uppercase, 1 special char
String passwordRegex = "^(?=.*[0-9])(?=.*[A-Z])(?=.*[!@#$%^&*]).{8,}$";
boolean validPassword = "Secure@123".matches(passwordRegex);     // true
```

Searching, Replacement, Splitting

```
"Hello World".matches("Hello.*");                    // true - full-string match
"Hello World".replaceAll("o", "0");                    // "Hell0 W0rld"
"a,b,,c".split(",");                                    // ["a", "b", "", "c"]
"The rain in Spain".replaceFirst("ain", "AIN");          // only first match replaced
```

Interview Questions — Part 10

Q1. Why compile a Pattern once instead of using String.matches() repeatedly? ⭐⭐⭐⭐

- **Ideal Answer:** `String.matches()` compiles the regex internally on every call, which is wasteful if the same pattern is used repeatedly (e.g., in a loop). Compiling a `Pattern` once and reusing the `Matcher` is significantly more efficient.
- **Difficulty:** Intermediate

Q2. Difference between `find()` and `matches()` on a Matcher? ⭐⭐⭐⭐

- **Ideal Answer:** `matches()` requires the **entire** input to match the pattern. `find()` searches for the **next** subsequence anywhere in the input that matches, and can be called repeatedly to find all matches.
- **Difficulty:** Intermediate

Part 11 — Reflection API ⭐⭐⭐⭐

What Reflection Is and Why It Exists

Reflection lets a program **inspect and manipulate classes, methods, fields, and constructors at runtime** — even ones it doesn't know about at compile time. It's how frameworks like **Spring** and **Hibernate** work their "magic": scanning classes for annotations, creating objects, and calling methods dynamically without you writing that wiring code yourself.

```
Class<?> clazz = Class.forName("com.example.User");    // load class by name at runtime
// or: Class<?> clazz = User.class;
// or: Class<?> clazz = someObject.getClass();

Object obj = clazz.getDeclaredConstructor().newInstance();   // create instance dynamically

Method method = clazz.getDeclaredMethod("setName", String.class);
method.setAccessible(true);        // bypass access checks (even for private methods!)
method.invoke(obj, "Riya");         // dynamically call setName("Riya")

Field field = clazz.getDeclaredField("name");
field.setAccessible(true);
System.out.println(field.get(obj));   // read a private field's value directly
```

Real-World Usage ⭐⭐⭐⭐⭐

Framework	How It Uses Reflection
Spring	Scans classes for `@Component` / `@Service` / `@Autowired`, creates beans, and injects dependencies — all via reflection
Hibernate/JPA	Reads `@Entity` / `@Column` annotations and maps object fields to database columns dynamically
JUnit	Discovers and invokes `@Test`-annotated methods without you registering them manually
JSON libraries (Jackson)	Reads/writes object fields dynamically to convert between JSON and Java objects

Advantages, Disadvantages, Performance ⭐⭐⭐⭐

Advantages	Disadvantages
Enables powerful frameworks (DI, ORM, testing tools)	**Slower** than direct code — bypasses JIT optimizations, has lookup overhead
Allows generic, reusable library code	**Breaks encapsulation** — can access private fields/methods, bypassing intended access control
Useful for tools like debuggers, serializers	**Security risk** — `setAccessible(true)` can violate a class's intended safety boundaries
	Errors move from compile-time to **runtime** (e.g., wrong method name → `NoSuchMethodException`)

Best practice: Application code should rarely use reflection directly — it's primarily a tool for **framework and library authors**, not everyday business logic.

Interview Questions — Part 11

Q1. What is Reflection and why do frameworks like Spring rely on it? ⭐⭐⭐⭐⭐ Frequently Asked

- **Ideal Answer:** Reflection allows inspecting and manipulating classes, methods, and fields at runtime. Spring uses it to scan for annotated classes, instantiate beans, and inject dependencies without the developer manually wiring everything together.
- **Difficulty:** Intermediate

Q2. What are the downsides of using Reflection? ⭐⭐⭐⭐

- **Ideal Answer:** It's slower than direct method calls (bypasses JIT optimization and does runtime lookups), it can break encapsulation by accessing private members, it introduces security risks, and it moves potential errors from compile-time to runtime.
- **Difficulty:** Intermediate

Part 12 — Annotations ⭐⭐⭐⭐

What Annotations Are

Annotations are metadata attached to code (classes, methods, fields) that don't directly change program logic, but can be read by the **compiler** or at **runtime** (typically via Reflection) to drive behavior — validation, configuration, code generation, dependency injection, etc.

Built-in Annotations

Annotation	Purpose
`@Override`	Tells the compiler this method should override a parent method — catches signature mistakes at compile time
`@Deprecated`	Marks an API as outdated; usage triggers a compiler warning
`@SuppressWarnings("...")`	Tells the compiler to ignore specific warnings (e.g., `"unchecked"`)
`@FunctionalInterface`	Documents/enforces that an interface has exactly one abstract method
`@SafeVarargs`	Suppresses warnings for potentially unsafe generic varargs

Meta-Annotations (used to build custom annotations) ⭐⭐⭐⭐

Meta-Annotation	Purpose
`@Retention`	Controls how long the annotation is kept: `SOURCE` (compiler only), `CLASS` (in bytecode, not runtime-visible), `RUNTIME` (accessible via Reflection)
`@Target`	Restricts where the annotation can be applied: `TYPE`, `METHOD`, `FIELD`, `PARAMETER`, etc.
`@Inherited`	Makes an annotation on a class automatically apply to its subclasses too

Writing a Custom Annotation

```
@Retention(RetentionPolicy.RUNTIME)   // must be RUNTIME to read it via reflection later
@Target(ElementType.METHOD)
@interface Loggable {
    String value() default "INFO";
}

class Service {
    @Loggable(value = "DEBUG")
    void process() { System.out.println("Processing..."); }
}

// Reading the annotation at runtime via Reflection
Method m = Service.class.getMethod("process");
if (m.isAnnotationPresent(Loggable.class)) {
    Loggable log = m.getAnnotation(Loggable.class);
    System.out.println("Log level: " + log.value());   // "DEBUG"
}
```

This exact pattern — a custom `RUNTIME`-retained annotation read via Reflection — is essentially how Spring's `@Transactional`, `@Cacheable`, and similar behavior-driving annotations work under the hood.

Interview Questions — Part 12

Q1. What is the difference between `@Retention(SOURCE)`, `CLASS`, and `RUNTIME`? ⭐⭐⭐⭐⭐

- **Ideal Answer:** `SOURCE` annotations are discarded by the compiler (e.g., `@Override`). `CLASS` annotations are stored in the `.class` bytecode but not accessible via reflection at runtime (rarely used directly). `RUNTIME` annotations are kept and can be read via Reflection while the program runs — required for anything a framework needs to inspect dynamically.
- **Why asked:** Directly tests whether the candidate understands how frameworks like Spring actually process annotations.
- **Difficulty:** Advanced

Q2. How does Spring know which classes to turn into beans just from annotations like `@Component`? ⭐⭐⭐⭐

- **Ideal Answer:** Spring scans the classpath at startup, uses Reflection to find classes annotated with `RUNTIME`-retained annotations like `@Component`, and instantiates/manages them as beans in its IoC container.
- **Difficulty:** Advanced

Part 13 — Frequently Confused Concepts ⭐⭐⭐⭐⭐

Checked vs Unchecked Exception

Checked	Unchecked
Subclass of `Exception` (not `RuntimeException`)	Subclass of `RuntimeException`
Compiler forces handling or declaration	No compiler enforcement
e.g. `IOException`, `SQLException`	e.g. `NullPointerException`, `ArithmeticException`

`throw` vs `throws`

`throw`	`throws`
Actually throws an exception instance	Declares a method might propagate a checked exception
Used inside method body	Used in method signature

String vs StringBuilder vs StringBuffer

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Thread-safe (by immutability)	Not synchronized	Synchronized
Slow for repeated modification	Fast, single-threaded use	Slower, multi-threaded safe

Array vs ArrayList (brief)

Array	ArrayList
Fixed size	Resizable
Stores primitives directly	Stores objects (autoboxing for primitives)
Faster, less overhead	More flexible, more overhead

Primitive vs Wrapper

Primitive	Wrapper
Stores actual value	Stores a reference to an object
Cannot be `null`	Can be `null` (NPE risk on unboxing)
Faster, less memory	Needed for collections/generics

Autoboxing vs Unboxing

Autoboxing	Unboxing
primitive → wrapper object (automatic)	wrapper object → primitive (automatic)
`Integer i = 5;`	`int x = i;`

Serialization vs Deserialization

Serialization	Deserialization
Object → byte stream	Byte stream → object
`ObjectOutputStream.writeObject()`	`ObjectInputStream.readObject()`

IO vs NIO

IO (`java.io`)	NIO (`java.nio`)
Stream-oriented, always blocking	Buffer/Channel-oriented, supports non-blocking
Simple, good for small/simple file tasks	Better for scalable, high-throughput I/O

Date vs LocalDate

`Date` (legacy)	`LocalDate` (java.time)
Mutable, not thread-safe	Immutable, thread-safe
0-indexed months	1-indexed months
Mixes date+time+zone	Cleanly separated types

Deep Copy vs Shallow Copy

Shallow Copy	Deep Copy
Copies top-level values; nested objects are shared references	Recursively copies nested objects too — fully independent
`clone()` default behavior	Requires manual/custom implementation

Enum vs Constants

Enum	`final` Constants
Type-safe, fixed set enforced by compiler	No type safety — any matching primitive/String is accepted
Can have fields, methods, constructors	Just a value, no behavior

Generic vs Object

Generic (`List<String>`)	Raw `Object` (`List`)
Compile-time type safety, no casting needed	Runtime `ClassCastException` risk, manual casting required

Part 14 — Java in Real Projects ⭐⭐⭐⭐

Domain	Where These Concepts Apply
Banking Applications	Custom checked exceptions (`InsufficientFundsException`); `BigDecimal` / `LocalDate` for money and dates (never `double` or legacy `Date`)
Hospital Management	Enums for `AppointmentStatus` ; `LocalDateTime` for scheduling; regex for validating patient IDs
Authentication Systems	Regex for password/email validation; `Optional` to avoid NPEs on user lookups; annotations (`@NotNull`) for validation
E-commerce	Generics in repository patterns; enums for `OrderStatus` ; Serialization for caching cart data
REST APIs	Custom exceptions mapped to HTTP status codes (`@ExceptionHandler`); annotations (`@RequestBody` , `@Valid`) processed via reflection
Spring Boot	Reflection + annotations are the backbone of the entire DI/IoC mechanism; generics in `JpaRepository<T, ID>`
File Upload Systems	NIO `Files` / `Path` API; byte streams for binary file handling; exception handling for I/O failures
Logging	String formatting/concatenation performance matters at scale; annotations like `@Loggable` (custom) or AOP-based logging
Configuration Files	Enums for environment types (`DEV` , `PROD`); Properties/YAML parsing relies on reflection-based binding
JSON Processing	Libraries like Jackson use Reflection + annotations (`@JsonProperty`) to map JSON ↔ Java objects automatically

OOP → Real Code Connection ⭐⭐⭐⭐⭐

```
@RestController                    Annotations processed via Reflection
class OrderController {
    @Autowired                     Dependency Injection (Reflection-based)
    private OrderService service;

    @PostMapping("/orders")
    ResponseEntity<Order> create(@RequestBody @Valid OrderRequest req) {
        try {
            return ResponseEntity.ok(service.create(req));
        } catch (InsufficientStockException e) {   Custom checked/unchecked exception
            return ResponseEntity.badRequest().build();
        }
    }
}
```

Nearly every Part in this handbook (exceptions, generics, enums, annotations, reflection) shows up together in a single typical Spring Boot controller — this is

exactly why interviewers test these topics individually before combining them in system-design-style questions.

Part 15 — Placement Focus Summary ⭐⭐⭐⭐⭐

Topic	Importance	Why Companies Ask
Checked vs Unchecked Exceptions	⭐⭐⭐⭐⭐	Foundational — tests whether you understand Java's error-handling philosophy
String Pool & Immutability	⭐⭐⭐⭐⭐	Extremely common "gotcha" questions (`==` vs `.equals()`)
StringBuilder vs String concatenation	⭐⭐⭐⭐⭐	Tests real-world performance awareness
Generics & PECS	⭐⭐⭐⭐⭐	Separates candidates with surface-level vs deep Java knowledge
Try-with-resources	⭐⭐⭐⭐⭐	Modern best practice — expected knowledge for any recent hire
Reflection + Annotations	⭐⭐⭐⭐	Bridges "how Java works" to "how Spring/Hibernate actually work"
Date/Time API	⭐⭐⭐⭐	Common in real business logic; legacy `Date` pitfalls are a classic trap question
Regex validation	⭐⭐⭐	Practical, hands-on coding-round topic
Serialization	⭐⭐⭐	Common in caching/session/distributed systems discussions

Part 16 — Common Java Mistakes

Mistake	Why It Happens	How to Avoid It
Improper exception handling (catching `Exception` broadly)	Convenience, laziness, or not knowing the specific exception type	Catch the most specific exception type; only broaden when genuinely necessary
Swallowing exceptions (`catch (Exception e) {}`)	"Making an error go away" without understanding it	Always log at minimum; never leave a catch block silently empty
Using `Exception` for control flow	Misusing exceptions as a "goto" mechanism	Use exceptions only for truly exceptional/error conditions, not routine logic
String comparison with `==`	Confusing reference and value comparison, especially due to String Pool caching small literals	Always use `.equals()` for content comparison
String concatenation inside loops	Not realizing each `+` creates a new String object	Use `StringBuilder` for repeated concatenation
Serialization mistakes (forgetting `serialVersionUID` , serializing sensitive fields)	Not understanding the serialization contract	Always declare `serialVersionUID` ; mark sensitive fields `transient`
Wrapper comparison mistakes (`==` on Integer)	Not knowing about Integer caching (-128 to 127)	Always use `.equals()` for wrapper value comparison
Incorrect generic usage (raw types, unchecked casts)	Mixing old pre-generics code with generic code	Avoid raw types entirely; heed "unchecked" compiler warnings
Reflection misuse	Using reflection in everyday business logic where normal method calls would do	Reserve reflection for framework/library-level code; prefer direct calls elsewhere
Regex mistakes (catastrophic backtracking, unescaped special characters)	Writing overly complex or careless patterns	Keep patterns simple; test with realistic and edge-case inputs; escape special characters
Date API misuse (using legacy `Date` in new code, ignoring timezones)	Old habits, copy-pasted legacy code	Always default to `java.time` classes; be explicit about `ZoneId` when it matters

```
// Common mistake: string concatenation in a loop
String result = "";
for (String item : items) {
    result += item + ", ";     // creates a new String object on every iteration!
}

// Fix:
StringBuilder sb = new StringBuilder();
for (String item : items) {
    sb.append(item).append(", ");
}
String result2 = sb.toString();
```

Part 17 — Final Revision

One-Page Quick Revision

Term	One-line Definition
Checked Exception	Compiler-enforced exception (must handle or declare) — e.g. `IOException`
Unchecked Exception	Runtime-only exception, not compiler-enforced — e.g. `NullPointerException`
Try-with-resources	Auto-closes `AutoCloseable` resources when the block exits
String Pool	Special memory area where string literals are cached and reused
String Immutability	Content never changes after creation; any "modification" returns a new object
StringBuilder	Mutable, fast, single-threaded string builder
Jagged Array	A multi-dimensional array whose rows can have different lengths
Integer Cache	`Integer` objects cached for values -128 to 127
Enum	Type-safe fixed set of constants, can have fields/methods/constructors
Generics	Compile-time type safety for classes/methods, avoiding manual casting
PECS	"Producer Extends, Consumer Super" — wildcard usage rule
Type Erasure	Generic type info is removed at compile time, replaced with `Object` /bound
Serialization	Converting an object into a byte stream
`transient`	Excludes a field from serialization
NIO	Buffer/Channel-based, non-blocking-capable I/O (vs stream-based blocking IO)
`java.time`	Immutable, thread-safe date/time API introduced in Java 8
Reflection	Runtime inspection/manipulation of classes, methods, fields
Annotation	Metadata attached to code, read by compiler or via Reflection
`@Retention(RUNTIME)`	Required for an annotation to be readable via Reflection at runtime

Exception Handling Cheat Sheet

- Hierarchy: `Throwable → Error / Exception → (Checked / RuntimeException)`
- `finally` always runs (except `System.exit()` or JVM crash)
- Multi-catch: `catch (IOException | SQLException e)`
- Try-with-resources auto-closes in **reverse** declaration order
- Custom exceptions: extend `Exception` (checked) or `RuntimeException` (unchecked)
- Never swallow exceptions silently — always log at minimum

String Cheat Sheet

- Literals go in the String Pool; `new String()` always creates a separate heap object
- `.intern()` manually pulls a string into the pool
- Use `.equals()` for content, never `==`
- `StringBuilder` for single-threaded repeated modification; `StringBuffer` for thread-safe

Generics Cheat Sheet

- `<? extends T>` = Producer (read-only safe)
- `<? super T>` = Consumer (write-only safe)
- Type erasure removes generic info at runtime — no generic arrays, no `instanceof` with parameterized types
- Avoid raw types — they defeat the purpose of generics entirely

IO & NIO Cheat Sheet

- `InputStream` / `OutputStream` = bytes; `Reader` / `Writer` = characters
- Always wrap with `Buffered*` for performance
- Try-with-resources for all closeable I/O resources
- NIO's `Path` / `Files` API is the modern default over the legacy `File` class
- `Files.lines()` streams lazily (memory-efficient); `readAllLines()` loads everything into memory

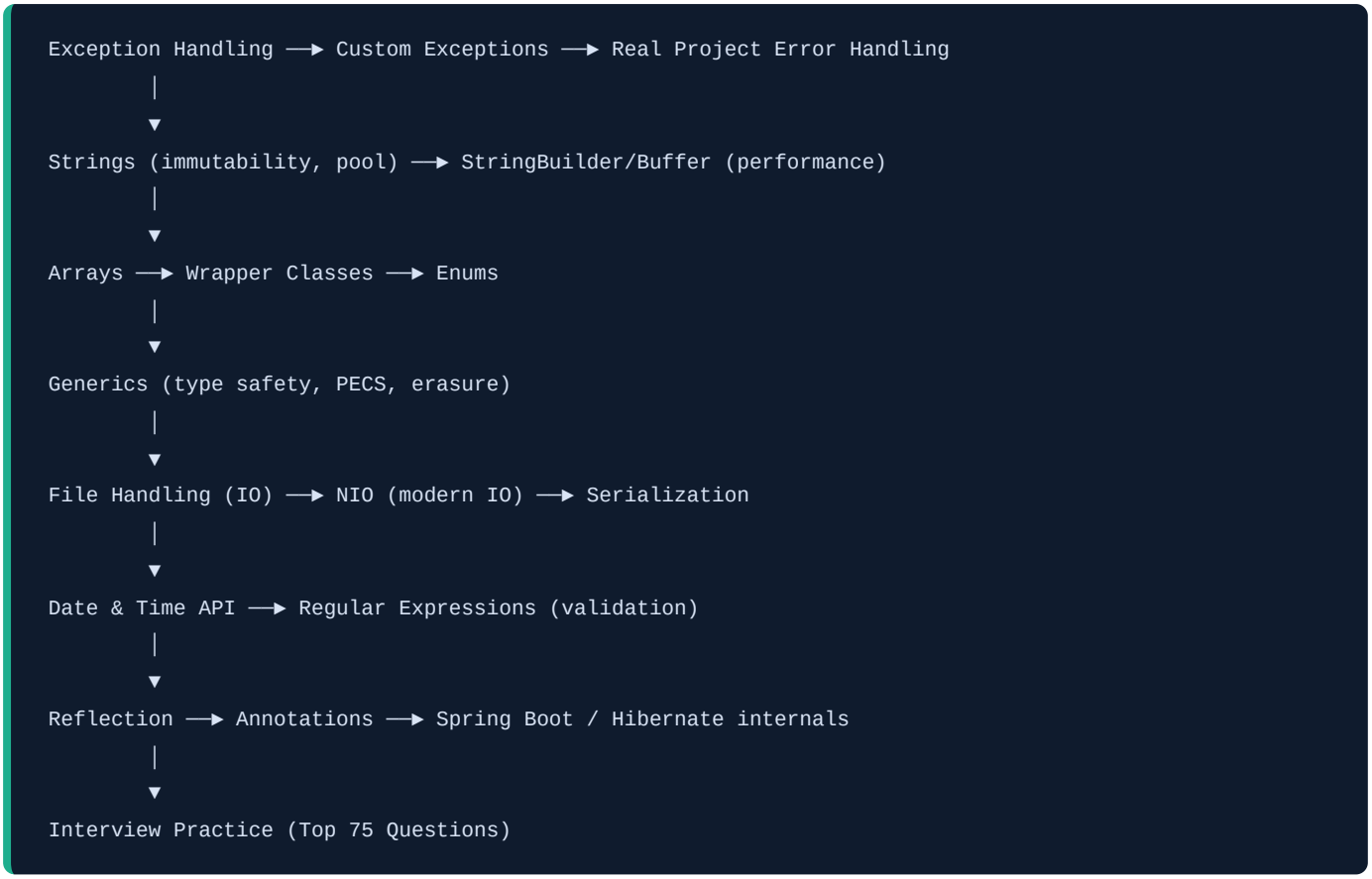
Date & Time Cheat Sheet

- `LocalDate` / `LocalTime` / `LocalDateTime` = no timezone; `ZonedDateTime` / `Instant` = timezone-aware
- `Period` = date-based amounts; `Duration` = time-based amounts
- Always prefer `java.time` over legacy `Date` / `Calendar`
- Months are 1-indexed in `java.time` (unlike legacy `Date`'s 0-indexed months)

Memory Tricks

- **Integer cache range:** "-128 to 127" — think "one byte's worth, roughly"
- **PECS:** "Producer Extends, Consumer Super"
- **String Pool:** literals are like a shared library book; `new String()` is like buying your own copy
- **transient:** "transient = temporary = don't save this field"

Topic Dependency Map



30-Minute Last-Minute Revision Checklist

- [ ] Can you explain checked vs unchecked exceptions with examples?
- [ ] Can you explain how try-with-resources works internally?
- [ ] Can you explain the String Pool and why `==` is unreliable for Strings?
- [ ] Can you explain when to use StringBuilder vs StringBuffer?
- [ ] Can you explain Integer caching and its `==` trap?
- [ ] Can you explain PECS with a code example?
- [ ] Can you explain type erasure and its consequences?
- [ ] Can you explain `transient` and `serialVersionUID`?
- [ ] Can you explain why `java.time` is better than legacy `Date`?
- [ ] Can you explain how Spring uses Reflection and Annotations together?

Top 75 Core Java Interview Questions

Exception Handling

1. **What is the exception hierarchy in Java?** — `Throwable` → `Error` / `Exception`; `Exception` → checked exceptions and `RuntimeException` (unchecked).
2. **Checked vs unchecked exceptions?** — Checked are compiler-enforced (`IOException`); unchecked are runtime-only, unenforced (`NullPointerException`).
3. **What is the difference between `throw` and `throws`?** — `throw` throws an instance; `throws` declares a method may propagate a checked exception.
4. **Does `finally` always run?** — Yes, except `System.exit()` or a JVM crash.

5. **What is try-with-resources?** — Automatically closes `AutoCloseable` resources when the block exits.
6. **What are suppressed exceptions?** — Exceptions thrown during auto-close that get attached to (not replace) the primary exception.
7. **When should you create a custom exception?** — When a specific, meaningful error condition isn't well represented by existing exception types.
8. **Should custom exceptions extend Exception or RuntimeException?** — `Exception` for recoverable conditions callers must handle; `RuntimeException` for programming errors.
9. **What happens if both try and finally throw exceptions?** — The finally block's exception wins; the try block's exception is lost.
10. **Why shouldn't you catch `Exception` broadly?** — It hides bugs and catches things you didn't intend to handle.

Strings

11. **Why are Strings immutable?** — Pool safety, thread safety, security, and hashcode caching.
12. **What is the String Pool?** — A special memory area caching string literals for reuse.
13. **`"abc" == new String("abc")` ?** — `false` — `new String()` bypasses the pool.
14. **What does `.intern()` do?** — Returns the pooled reference for an equal string, adding it to the pool if absent.
15. **StringBuilder vs StringBuffer?** — StringBuilder is faster/unsynchronized; StringBuffer is synchronized/thread-safe.
16. **Why avoid `+=` in loops?** — Each concatenation creates a new String object — use StringBuilder instead.
17. **Is String thread-safe?** — Yes, because it's immutable.
18. **How is String's hashCode cached?** — Computed once since content never changes, then reused on subsequent calls.

Arrays

19. **Default value of an int array element?** — `0`.
20. **Default value of an object array element?** — `null`.
21. **Arrays.equals() vs Arrays.deepEquals()?** — `equals()` is shallow (one level); `deepEquals()` recursively compares nested arrays.
22. **Is array cloning a deep copy?** — No, it's shallow — object arrays share the same inner object references.
23. **What is a jagged array?** — A multi-dimensional array where rows can have different lengths.

Wrapper Classes

24. **What is autoboxing?** — Automatic conversion of a primitive to its wrapper class.
25. **Why does `Integer a=200,b=200; a==b` return false?** — 200 is outside the Integer cache range (-128 to 127).
26. **What causes NPE during unboxing?** — Calling `.intValue()` (implicitly) on a `null` wrapper reference.

Enums

27. **Why use enums over int constants?** — Type safety, ability to hold fields/methods, `switch` support.
28. **Can enum constructors be public?** — No, they must be private/package-private.
29. **What is EnumSet backed by internally?** — A bit vector, for fast, compact operations.

Generics

30. **Why were generics introduced?** — For compile-time type safety, avoiding manual casts and runtime ClassCastExceptions.
31. **What is type erasure?** — The compiler removes generic type info at compile time, substituting Object/the bound type.
32. **What is PECS?** — "Producer Extends, Consumer Super" — the rule for choosing wildcard bounds.
33. **Can you create a generic array directly?** — No, due to type erasure.
34. **What is heap pollution?** — When an object of the wrong parameterized type ends up in a generic structure via raw types/unchecked casts.
35. **What is a bounded type parameter?** — A generic type restricted to a specific type or its subtypes, e.g. `<T extends Number>`.

File Handling & NIO

36. **Byte streams vs character streams?** — Byte streams handle raw binary data; character streams handle text with encoding awareness.
37. **Why wrap with BufferedReader/Writer?** — Reduces the number of actual I/O system calls, improving performance.
38. **What does `transient` do?** — Excludes a field from serialization.
39. **What is serialVersionUID for?** — Version compatibility check during deserialization.
40. **Serializable vs Externalizable?** — `Serializable` is a marker interface with default serialization; `Externalizable` gives full manual control over the format.
41. **Why was NIO introduced?** — To support buffer/channel-based, non-blocking I/O for scalable applications.
42. **Path/Files vs the old File class?** — Path/Files (NIO.2) is more powerful, supports streaming, and is the modern standard.
43. **Files.lines() vs Files.readAllLines()?** — `lines()` streams lazily (memory-efficient); `readAllLines()` loads the whole file into memory.

Date & Time

44. **Why avoid java.util.Date?** — It's mutable, not thread-safe, and has confusing 0-indexed months.
45. **Duration vs Period?** — Duration = time-based amounts; Period = date-based amounts.
46. **LocalDateTime vs ZonedDateTime?** — LocalDateTime has no timezone; ZonedDateTime includes one.
47. **Is LocalDate thread-safe?** — Yes, because java.time classes are immutable.

Regex

48. **Pattern vs Matcher?** — Pattern is the compiled regex; Matcher performs match operations against specific input.
49. **find() vs matches()?** — `matches()` requires the whole input to match; `find()` searches for the next matching subsequence.
50. **Why compile a Pattern once?** — Avoids recompiling the regex on every call, improving performance in loops.

Reflection & Annotations

51. **What is Reflection?** — Runtime inspection/manipulation of classes, methods, fields, and constructors.
52. **Why does Spring rely on Reflection?** — To scan annotated classes and wire dependencies without manual registration.
53. **Downsides of Reflection?** — Slower execution, breaks encapsulation, security risk, runtime (not compile-time) errors.
54. **What does setAccessible(true) do?** — Bypasses Java's access control checks, allowing access to private members.
55. **@Retention(RUNTIME) vs SOURCE vs CLASS?** — Only RUNTIME-retained annotations are readable via Reflection while the program runs.
56. **What is a marker annotation?** — An annotation with no elements, used purely to tag code (similar concept to marker interfaces).
57. **How would you write a custom annotation read at runtime?** — Define it with `@Retention(RUNTIME)`, apply `@Target`, then read it via `Method.isAnnotationPresent()` / `getAnnotation()`.

Mixed / Scenario-Based

58. **Scenario: A HashMap lookup with an Integer key sometimes fails unexpectedly after `==` comparison — why?** — Comparing via `==` on Integer objects outside the -128 to 127 cache range fails; always use `.equals()` /rely on HashMap's own equals-based lookup.
59. **Scenario: Reading a huge log file crashes with OutOfMemoryError — what's the fix?** — Use `Files.lines()` (lazy streaming) instead of loading the whole file via `readAllLines()`.
60. **Scenario: Deserializing an old saved object throws InvalidClassException — why?** — The class's `serialVersionUID` no longer matches; the class structure changed since serialization.
61. **Scenario: A method silently returns `null` and a NullPointerException occurs three calls later — how do you prevent this?** — Fail fast with early null checks or use `Optional` to make the possibility of absence explicit.
62. **Scenario: String concatenation in a loop is causing performance issues in production — what's the fix?** — Replace with `StringBuilder.append()`.
63. **Scenario: A List<? extends Number> throws a compile error when you try to add to it — why?** — The wildcard's actual type is unknown; adding isn't type-safe, so it's disallowed except for `null`.

64. **What is the difference between compile-time and runtime polymorphism, revisited in Core Java context?** — Overloading resolves at compile time; overriding resolves at runtime via dynamic dispatch.
65. **Why might catching `Throwable` instead of `Exception` be dangerous?** — It can catch `Error`s like `OutOfMemoryError`, which typically shouldn't be handled as recoverable conditions.
66. **What's a real risk of using reflection-heavy frameworks in performance-critical code paths?** — Reflection overhead (method lookup, bypassed JIT optimization) can add latency in hot paths.
67. **Why is immutability valuable beyond just Strings?** — Immutable objects are inherently thread-safe, simpler to reason about, and safe to cache/share.
68. **What is the purpose of `Optional` in modern Java?** — To make the possibility of a missing value explicit in the type system, reducing accidental NPEs.
69. **How does `HashMap` use `hashCode()` and `equals()` together?** — `hashCode()` determines the bucket; `equals()` confirms identity within that bucket for lookups/insertions.
70. **What is the benefit of `EnumMap` over `HashMap` for enum keys?** — Internally array-backed by ordinal, giving faster access and more compact memory usage.
71. **Why does `Arrays.asList()` return a fixed-size list?** — It's backed directly by the underlying array, so structural changes (add/remove) aren't supported, only element updates.
72. **What's the danger of using raw types like `List` instead of `List<String>`?** — Loses compile-time type checking, reintroducing the risk of runtime `ClassCastException`.
73. **How would you validate a password using regex?** — Use lookahead assertions to enforce multiple conditions (digit, uppercase, special character) within a single pattern.
74. **Why is `BigDecimal` preferred over `double` for financial calculations?** — `double` uses binary floating-point representation, which can't precisely represent many decimal fractions, causing rounding errors unacceptable in financial contexts.
75. **How do annotations, generics, and reflection combine in a typical Spring Boot app?** — Annotations mark intent (`@Component`, `@Autowired`); generics provide type-safe repositories (`JpaRepository<T,ID>`); reflection scans and wires it all together at startup.

Key Takeaways

- **Exceptions exist to separate error handling from normal logic** — always catch the most specific type, and never swallow silently.
- **Strings are immutable for good reasons** (pooling, thread safety, security) — but that means loops need `StringBuilder`, not `+=`.
- **Generics move type errors from runtime to compile time** — understand PECS and type erasure to reason about them confidently.
- **Reflection + Annotations are the backbone of Spring/Hibernate** — understanding them demystifies "framework magic."
- **Always prefer `java.time` over legacy `Date`**, and prefer NIO's `Path` / `Files` over the legacy `File` class in new code.

End of Handbook 2. Continue to Handbook 3 (Collections Framework & Multithreading) to keep building your placement-ready Java foundation.